# Self-Healing Network Operator for Kubernetes

## Pod-to-Pod Data Plane Connectivity via Synthetic Probing

Nilay Verma
*Roll No. 23110219*

Siddhesh Umarjee
*Roll No. 23110347*

Patel Ridham Vijaykumar
*Roll No. 23110238*

Eshaan Ashish Jaiswal
*Roll No. 24110116*

Anurag Singh
*Roll No. 24110061*

### I. Introduction

Kubernetes keeps applications available by restarting containers that fail liveness probes and by removing pods that fail readiness probes from Service endpoints [1]. Both checks are local: they ask whether a container answers, not whether a pod on one node can reach a pod on another. East-west traffic between pods crosses the CNI data plane (virtual interfaces, routes, encapsulation, iptables rules and NetworkPolicy), and any of these can fail while every pod still reports Running and Ready. Such silent failures surface as application timeouts long before the platform names a cause.

This project builds a Kubernetes operator that continuously tests the pod network itself. A probe agent runs on every node; a manager asks each agent to probe every other agent over TCP, assembles a directed reachability matrix, classifies the shape of the failures (for example, one node's outgoing row failing versus its incoming column) and publishes the verdict as the status of a NetworkHealthCheck custom resource and as Prometheus metrics. The idea follows active-probing systems for datacenter networks such as Pingmesh [2] and NetBouncer [3], scaled down to one cluster and exposed through the Kubernetes API. Our main contributions are:

- A rule-based classifier that localizes failures from the directed matrix into ten classes, including **NoData** (nothing was measured, so the system never reports health without evidence) and **Degraded** (reachable, but with at least 5% probe loss).
- Per-pair hysteresis in the live reconcile path (three failed rounds to raise an alarm, two good rounds to clear it), so one bad round cannot flip the verdict.
- A bidirectional TCP probe (payload plus a 1-byte acknowledgement) served by an agent API that requires a token and refuses targets outside the cluster networks.
- Bounded Kubernetes status (at most 500 matrix entries plus full-matrix counts), and Prometheus metrics and a dashboard that a unit test checks against the code.
- A reproducible KIND testbed with node-level fault injection, and a 60-trial evaluation in which all five fault types and a healthy control were classified correctly.

### II. Problem Statement

Consider a cluster with one agent on each of $N$ schedulable nodes. A probe round measures every ordered pair $(i, j)$, $i \neq j$, and produces a directed matrix $M$ with $N(N-1)$ entries. Each entry $M[i, j]$ records whether at least one of $k$ probes completed a bidirectional exchange, the loss rate (failed probes / $k$), the mean, minimum and maximum RTT, the jitter, the longest run of lost probes, and whether the control request itself failed. The problem is to compute, every round, a verdict $f(M)$ = (class, confidence, suspect nodes) that (a) never claims health without evidence, (b) separates a hard failure from partial degradation, (c) localizes a failure to a pair, to one direction of a node, to a node or to the whole mesh, and (d) does not change on a single noisy round.

**Inputs:** a cluster-scoped NetworkHealthCheck object (probe interval, timeout, count and payload size, failure and success thresholds, concurrency limit, remediation flags) and the set of running agent pods. **Outputs:** the object's status (verdict, confidence, suspect nodes, a bounded sample of matrix entries, pair counts and a Healthy condition) and Prometheus gauges. **Assumptions:** agents reach each other by pod IP, so a fault that blocks the agents' data port also blocks workload traffic on that path; and when the manager cannot reach an agent, that is a measurement problem, reported as Unknown rather than as a network fault. **Constraints:** status must stay far below etcd's request-size limit of about 1.5 MiB [4]; the probe API must not become an open scanner; and the operator must never take destructive action on its own. **Scope:** TCP reachability and quality between pods on different nodes, failure localization, observability and evaluation are in scope. DNS, Service load balancing, application-level health, automatic repair and multi-cluster operation are out of scope.

TABLE I
Existing approaches and the gap

| Approach | What it checks | Gap for this problem |
|---|---|---|
| Liveness / readiness probes [1] | A container answers on its own node | No pod-to-pod path across nodes is tested |
| Node conditions | Kubelet and node resource health | The network can fail while every node is Ready |
| Goldpinger [5] | DaemonSet agents call each other; graph and metrics | Detects failures but does not classify their shape or publish a Kubernetes verdict |
| Prometheus blackbox exporter [6] | Synthetic probes to configured targets | One vantage point, so one-way faults stay invisible |
| Service-mesh telemetry | Real application traffic | Passive: idle paths are never tested; needs sidecars |

The gap is a lightweight, Kubernetes-native check that tests every directed pod-to-pod path, says which node and direction

failed, refuses to report health without data, and is safe to run unattended.

## III. Project Objectives

We set eight objectives, each with a pass condition that is checked in Sections V–VII:

- **O1 – Detection and localization:** each injected fault type (F2–F6) is detected and given its expected class in at least 9 of 10 trials.
- **O2 – No false alarms:** no unhealthy transition during 10 healthy-cluster control trials (F1).
- **O3 – Evidence before health:** an empty matrix yields NoData with confidence 0, never Healthy.
- **O4 – Quality vs. reachability:** a reachable path with at least 5% probe loss is reported as Degraded (scenario F5, 40% loss).
- **O5 – Stability:** a single failed round does not change the verdict; recovery needs two consecutive good rounds.
- **O6 – Bounded state:** status holds at most 500 matrix entries at any cluster size (tested with 600 pairs).
- **O7 – Safe operation:** the probe API returns 401 without a token and 403 for off-allowlist or cloud-metadata targets, and remediation never deletes pods.
- **O8 – Reproducibility:** the testbed, the experiments and the summary tables are rebuilt from make targets and scripts in the repository.

The non-functional requirements are measured as follows. **Performance:** time to detect a fault (MTTD), probe RTT and round duration. **Simplicity:** the number of deployed components, and the probes and bytes per round. **Robustness:** loss, jitter and loss bursts under faults, the false-positive rate, and how the system behaves when the measurement itself fails.

## IV. Architecture and Design

### A. Components

Fig. 1 shows the system. It has four runtime parts and an optional monitoring stack:

- **NetworkHealthCheck CRD** (cluster-scoped, API group network.selfheal.io, version v1alpha1). The spec holds every tunable and the status holds the result, including one Healthy condition in the style of the Kubernetes API conventions [7]. Every spec field changes behaviour; fields that did nothing (`topology`, `sampleDegree`, the HTTP and ICMP probe types) were removed.
- **Manager** (a Deployment with one replica). A polling controller that, for each NetworkHealthCheck whose interval has elapsed, discovers agents, runs a probe round, applies hysteresis, classifies the result, writes status, and serves `/healthz` and `/metrics`.
- **Agent** (a DaemonSet). A control API on port 8080 (`/probe`, `/healthz`, `/metrics`) and a separate TCP data listener on port 9376 that answers every connection with a 1-byte acknowledgement.
- **Analysis library** (`pkg/analysis`). A pure classifier that never reads the clock, and a per-pair hysteresis tracker; both are tested against 13 fixture matrices.
- **Monitoring.** Prometheus scrapes the manager and the agents, and a provisioned Grafana dashboard shows the verdict, classification, confidence, pair states, RTT and loss.

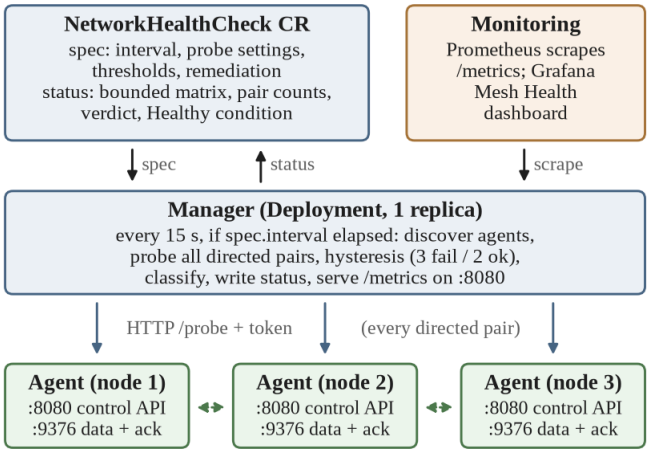



Fig. 1. System architecture. The manager only orchestrates; agents send the actual data-plane probes to each other over the pod network.

### B. Reconcile Round

Every 15 s the manager lists NetworkHealthCheck objects and runs a round only for objects whose `status.lastProbeTime` is older than `spec.interval`. A round (i) lists running agent pods that have a pod IP; (ii) for every ordered pair asks the source agent, over HTTP with a bearer token, to probe the destination pod IP, with at most `maxConcurrentProbes` requests in flight; (iii) builds the directed matrix; (iv) updates the hysteresis tracker and derives a stabilized matrix; (v) classifies it; (vi) writes the status, retrying on update conflicts, and records metrics; and (vii) computes an alert-only remediation plan. If the HTTP request to an agent fails, that pair is marked as an agent error, which the classifier turns into Unknown instead of a data-plane verdict.

### C. Classification

A pair fails when none of its probes completes. The classifier reads the shape of the failures (Fig. 2): a failed row means a node cannot send, a failed column means it cannot receive, and both mean it is isolated. The rules in Table II are checked from top to bottom, so hard failures take precedence over partial loss. For the node-level classes, confidence is the fraction of all failed pairs that the hypothesis explains; a hypothesis is reported only if it reaches the 0.80 confidence floor, otherwise the verdict is Unknown. Remediation planning ignores verdicts below 0.8 confidence.

TABLE II
Classification rules (checked top to bottom)

| Class | Matrix evidence | Confidence |
|---|---|---|
| NoData | Empty matrix (no agents or pairs) | 0 |

| Class | Matrix evidence | Confidence |
|---|---|---|
| Unknown | Any agent error (checked next); also the fallback when no rule below applies | 0, or failed share |
| Healthy | No failed pair; every loss < 5% | 1.0 |
| Degraded | No failed pair; some loss ≥ 5% | 1.0 |
| PairLocalFailure | Exactly one directed pair fails | 1.0 |
| NodeIsolated | One node's whole row and column fail | explained / failed |
| NodeEgressFailure | One node's whole row fails | explained / failed |
| NodeIngressFailure | One node's whole column fails | explained / failed |
| PolicyScopedFailure | Every pair fails with "connection refused" | 1.0 |
| ClusterPartition | At least 80% of pairs fail | failed / total |

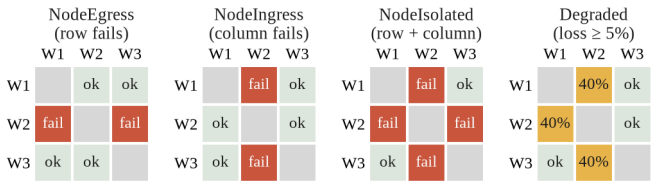



Fig. 2. Matrix shapes the classifier recognizes. Rows are sources, columns are destinations, and W2 is the faulty node.

### *D. Design Decisions and Trade-offs*

- **Distributed agents rather than a central prober.** A single prober only sees paths from its own node and misses one-way faults. Agents cost one small pod per node (64 MiB memory limit) but test exactly the paths that workloads use.
- **Separate control and data ports.** Keeping the control API (8080) apart from the probed port (9376) lets the system tell "could not ask the agent" (Unknown) from "the agent's probe failed" (a data-plane fault), and lets experiments break the data path alone.
- **Full mesh rather than sampling.** Probing all $N(N-1)$ pairs gives complete row and column evidence for localization at $O(N^2)$ cost: 6 pairs for 3 nodes but 506 for 23. Sampling would scale better but weakens the row and column rules, so it is future work.
- **Rules rather than learning.** A deterministic classifier is explainable, can be tested against fixture matrices and needs no training data.
- **Hysteresis: latency vs. stability.** Requiring three consecutive failed rounds delays alarms (Section VII) but stops a single lost round from changing the verdict. The thresholds are spec fields, so operators can choose the trade-off.
- **Bounded status rather than the full matrix in etcd.** The matrix grows as $O(N^2)$ and is rewritten every round, while etcd rejects requests above about 1.5 MiB [4] and every write is sent to all watchers. Status therefore keeps the first 500 entries in sorted order, the total pair count, a truncation flag, pair counts by state over the full matrix and up to 50 degraded pairs; per-pair detail goes to metrics.
- **Polling rather than a watch-based controller.** A 15 s poll gated by each object's interval is easy to reason about, at the cost of some detection latency.
- **Alert-only remediation.** An early version "remediated" by deleting agent pods. That restarts the measurement, not the network, and could erase an injected fault so that detection looked like healing. Every network class now maps to an alert, and no action is executed.
- **TCP with an acknowledgement rather than ICMP.** A TCP connect proves only that the handshake passed, so the agent writes a payload and then reads a 1-byte application acknowledgement, which proves the return path. TCP also needs no NET_RAW capability; the agent drops NET_RAW and adds no capabilities.

## V. Implementation

Both binaries are written in Go 1.22 with k8s.io/client-go: a typed client for pods and a dynamic client for the custom resource, without controller-runtime. Metrics are written in the Prometheus text format by hand. Images are built with multi-stage Dockerfiles (distroless for the manager, Alpine for the agent) and deployed with Kustomize. The code is organized as `cmd/` (entry points), `internal/controller/` (poll loop, reconcile, probe client, status builder, metrics, remediation), `pkg/analysis/` (classifier and hysteresis), `pkg/prober.go` and `pkg/server.go` (agent), `config/` (CRD, RBAC, manifests, monitoring) and `hack/` (cluster setup, fault injection, measurement). Algorithm 1 summarizes one round. The 47 unit tests in three packages cover the classifier fixtures, hysteresis, the probe acknowledgement, authentication and target validation, status truncation, metrics and remediation safety.

**Algorithm 1. One reconcile round**

```
if now - lastProbeTime < interval: return
A <- running agent pods with a pod IP
for each (s, d) in A x A, s != d, in parallel
    (at most maxConcurrentProbes at once):
  M[s->d] <- GET s:8080/probe?target=d.podIP
    # k probes: dial (2 s), write payload,
    # read 1-byte ack; reachable if any ack
  on HTTP error: M[s->d].agentError <- true
tracker.update(M)          # 3 fails, 2 oks
M' <- stabilize(M, tracker)
v  <- classify(M')
write bounded status(v, M'); record metrics
plan AlertOnly (never executed)

classify(M):
  M empty              -> NoData (conf. 0)
  any agentError       -> Unknown
  F <- pairs with no completed probe
  F empty: any loss >= 5% -> Degraded,
           else           -> Healthy
  |F| = 1              -> PairLocalFailure
  n: row+col failed    -> NodeIsolated
  n: row failed        -> NodeEgressFailure
  n: column failed     -> NodeIngressFailure
    (node rules need >= 80% of F explained)
  all pairs refused    -> PolicyScopedFailure
  |F| >= 80% of M      -> ClusterPartition
  otherwise            -> Unknown
```

### A. Probe Protocol and Agent Security

For each of $k$ probes (5 by default) the agent dials the target's data port with the probe timeout (2 s), sets one deadline for the write and the read, writes the payload (1024 B) and reads exactly one byte. A probe succeeds only if the acknowledgement arrives, so a listener that accepts and immediately closes counts as a failure. Probes to one target run one after another and yield the loss rate, mean, minimum and maximum RTT, jitter (the standard deviation of successful RTTs), the longest loss burst and a synthetic throughput. A pair is reachable if at least one probe completed; path quality is carried by the loss rate. The `/probe` endpoint requires a bearer token (`PROBE_TOKEN` on the agent and `AGENT_PROBE_TOKEN` on the manager, both from one Secret), accepts only IP literals, and answers 403 for loopback, link-local (including 169.254.169.254), multicast, unspecified or off-allowlist addresses. HTTP server timeouts bound slow clients.

### B. Challenges and Solutions

- **Healthy without evidence.** The first classifier returned Healthy with confidence 1.0 for an empty matrix, for example when no agents were running. It now returns NoData with confidence 0, and remediation ignores it.
- **Heavy loss looked healthy.** Because a pair is reachable if any probe succeeds, 40% loss stayed Healthy and F5 always timed out. A Degraded class with a configurable 5% loss threshold fixed this.
- **False success from a bare connect.** A probe that only completed the TCP handshake succeeded against a listener that closed immediately. Reading the acknowledgement closed this gap; a test server that accepts and closes now fails the probe.
- **An open probe endpoint.** `/probe` accepted any target, which turned every agent into an SSRF and port-scanning tool inside the cluster. The token and the address allowlist bound it, and tests check the 401 and 403 responses.
- **Fields that did nothing or vanished.** Several spec fields were parsed but ignored, and new status fields were silently dropped by the API server because the CRD's structural schema did not declare them [8]. Unused fields were removed and every status field was declared.
- **Remediation that hid faults.** Deleting agent pods as "self-healing" also cleared faults that had been injected inside those pods. Remediation became alert-only, and faults are now injected on the KIND node itself (`docker exec`), where a pod restart cannot clear them.
- **Dashboard and documentation drift.** The dashboard queried metrics that were never emitted. A unit test now fails if the README or the dashboard names a metric that the manager does not export.
- **Testbed decay.** On a long-lived KIND cluster, Calico's CNI token expired and pods on one node could not start. Restarting that node's calico-node pod issues a fresh token.

The code, manifests, scripts and raw results are at https://github.com/nilay-v3rma/CS331-T011-KubernetesSelfHealing.

## VI. Testbed Setup

All experiments ran on one host with a KIND [9] cluster, whose nodes are Docker containers, using Calico [10] as the CNI. Table III lists the environment and every parameter; highlighted cells are details of the evaluation host that are still to be filled in. Table IV lists the fault scenarios. All faults target the worker node `pod-pod-selfheal-worker2`, and each injection is undone after its trial.

TABLE III
Testbed and configuration

| Item | Value |
|---|---|
| Host CPU | AMD Ryzen 7 5800H with Radeon Graphics, 3201 MHz, 8 Core(s), 16 Logical Processor(s) |
| Host RAM | 16.0 GB Installed Physical Memory |
| Host OS | Microsoft Windows 11 Home Single Language Version 10.0.26300 Build 26300 |
| Docker Version | 29.9.2 |
| Cluster | KIND v0.23.0, Kubernetes v1.30.x; 1 control-plane + 3 worker nodes |
| CNI | Calico v3.28.0 (KIND default CNI disabled); pod CIDR 192.168.0.0/16 |
| Software | Go 1.22 and client-go; manager and agent images built locally and loaded into KIND |
| Workload | 3 agents (one per worker), 6 directed pairs; 5 probes of 1024 B per pair, so 30 probes and up to 30 KB per round |
| Probing | interval 15 s; manager poll 15 s; probe timeout 2 s; maxConcurrentProbes 64 |
| Analysis | failure/success thresholds 3/2; degraded loss threshold 5%; confidence floor 0.80 |
| Status limits | 500 reachability entries; 50 degraded pairs |
| Measurement | status polled every 1 s; detection timeout 120 s; F1 observed for 45 s; 10 trials per scenario |

TABLE IV
Fault scenarios (injected on worker2)

| ID | Injection | Expected class |
|---|---|---|
| F1 | None (healthy control) | Healthy |
| F2 | iptables DROP of inbound TCP 9376 (INPUT, and FORWARD to the agent) | NodeIngressFailure |
| F3 | iptables DROP of outbound TCP 9376 (OUTPUT, and FORWARD from the agent) | NodeEgressFailure |
| F4 | DROP of TCP 9376 in both directions and forced deletion of the node's calico-node pod | NodeIsolated |
| F5 | tc netem 40% packet loss on the node's default-route interface [11] | Degraded |
| F6 | Deny-all ingress and egress NetworkPolicy in the agent namespace | ClusterPartition |

## VII. Evaluation

### A. Methodology

For each trial, `hack/measure-mttd.sh` clears any earlier fault, waits until the NetworkHealthCheck reports Healthy, records the injection time, injects the fault and polls the status every second. A detection is the first Healthy=False condition whose transition time is after the injection. MTTD is the time from injection to that observation, and the predicted class and matrix averages are read from the same status. A trial with no detection within 120 s is recorded as a timeout; none occurred. In the F1 control nothing is injected and the status is watched for 45 s; any unhealthy transition counts as a false positive. Each scenario ran 10 trials (60 in total), and `hack/summarize-results.py` builds the tables below from the raw CSV file.

### B. Detection Accuracy and Latency

All 60 trials were classified correctly: every fault was detected with its expected class, so the confusion matrix is purely diagonal, and the 10 healthy controls raised no false alarm (Table V). Objectives O1 and O2 are therefore met.

TABLE V
Detection results (MTTD in seconds)

| Scen. | N | Correct | Median | Mean ± sd | Range | v1 mean |
|---|---|---|---|---|---|---|
| F1 | 10 | 10 (0 FP) | – | – | – | – |
| F2 | 10 | 10 | 85.2 | 85.0 ± 0.8 | 82.9–85.5 | 15.1 |
| F3 | 10 | 10 | 85.1 | 85.0 ± 0.4 | 83.9–85.2 | 13.0 |
| F4 | 10 | 10 | 84.8 | 85.1 ± 0.7 | 84.4–86.2 | 18.6 |
| F5 | 10 | 10 | 20.7 | 25.8 ± 12.1 | 18.7–49.3 | missed[a] |
| F6 | 10 | 10 | 84.7 | 85.0 ± 0.6 | 84.6–86.1 | 14.7 |

[a]Not detected within the 120 s timeout in any of the 3 v1 trials.

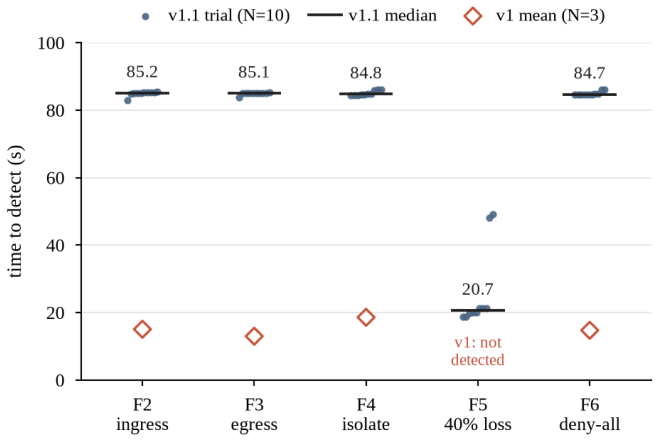



Fig. 3. Time to detect each fault. Dots are v1.1 trials, bars are v1.1 medians, and diamonds are the v1 baseline means (3 trials each).

Hard faults (F2, F3, F4 and F6) were detected in about 85 s, with a spread of under 1 s. This latency comes from the design, not from noise. The harness injects just after a round completes, because it waits for Healthy=True, which is written at the end of a round; the first round after the fault therefore starts at the next 15 s tick. In that round every probe on a dead path waits for the 2 s dial timeout, and the five probes of a pair run one after another, so the round takes about 10 s. Because the interval gate measures from the end of the previous round, the tick 5 s later is skipped and rounds fall to a 30 s period. Three failed rounds are needed before the verdict changes: 15 + 30 + 30 + 10 ≈ 85 s, which matches every hard-fault trial.

F5 needs no failure hysteresis. Lossy pairs still complete some probes, so they remain reachable, and the first round that measures at least 5% loss on any pair yields Degraded, at a median of 20.7 s. In two trials (48.1 s and 49.3 s), TCP retransmission hid the loss in the first round: every probe completed, only more slowly, and the next round ran one 30 s period later. F5 was classified as Degraded in all 10 trials, compared with 0 of 3 for v1, which meets O4.

The earlier v1 (no hysteresis, 3 trials per scenario) detected hard faults faster, in 13–19 s, but never detected F5, could report Healthy with no data, and could be flipped by a single bad round. The roughly 70 s difference is the explicit price of the three-round failure threshold and the timeout-stretched round period. Both are configurable (Section VIII).

### C. Non-Functional Measurements

TABLE VI
Probe metrics in the detecting round (means of 10 trials)

| Scen. | RTT (ms) | Jitter (ms) | Loss | Max burst | MB/s | Bytes |
|---|---|---|---|---|---|---|
| F2 | 0.404 | 0.191 | 0.333 | 5 | 1.17 | 20,480 |
| F3 | 0.596 | 0.604 | 0.333 | 5 | 1.16 | 20,480 |
| F4 | 0.137 | 0.025 | 0.667 | 5 | 0.71 | 10,240 |
| F5 | 292.7 | 301.6 | 0.073 | 2 | 0.67 | 28,467 |
| F6 | 0.000 | 0.000 | 1.000 | 5 | 0.00 | 0 |

**Performance.** On healthy paths RTT is below a millisecond (0.1–0.6 ms, since all nodes share one host), and a healthy round of six pairs takes a fraction of a second (0.07 s on our development cluster), so probing overhead is negligible and MTTD is set by the thresholds above. Under F5 the mean RTT of successful probes rises to about 293 ms with about 302 ms of jitter, because TCP retransmission timers recover the lost segments; the system reports this loss of quality even though every pair stays reachable. **Simplicity.** The deployment is one manager, one DaemonSet and one CRD. A round costs 30 probes and at most about 30 KB of synthetic traffic for three nodes, growing as $5N(N-1)$ probes. **Robustness.** The loss rates match the geometry of each fault exactly: one row or column of six pairs (1/3) for F2 and F3, a row plus a column (2/3) for F4, and every pair for F6. The burst metric separates complete per-pair outages (5 of 5 probes lost) under hard faults from short bursts (at most 2) under random loss. Failures of the measurement itself are reported as Unknown or NoData, not as network faults.

### D. Threats to Validity

The cluster is emulated on a single host, so latency and throughput do not represent a physical network, and netem acts on a container interface. With 10 trials per scenario we report ranges, not statistical significance. All faults target one

worker node. The F5 loss applies to all traffic on the node's interface, not only to the probe port. The F1 control observed 45 s per trial (7.5 min in total), so rare false positives could be missed. MTTD includes up to 1 s of polling granularity. Only detection is measured; recovery time (MTTR) is not, because remediation is alert-only.

## VIII. Summary

This project addressed silent pod-to-pod connectivity failures in Kubernetes, where every pod is Running and Ready but the data plane between nodes drops or degrades traffic. Per-node agents actively probe every directed path with an acknowledged TCP exchange, a manager classifies the resulting matrix by its failure shape, and the verdict is published as bounded Kubernetes status and as Prometheus metrics. On a four-node KIND cluster with Calico, all five fault types and the healthy control were classified correctly across 60 trials. Partial loss is reported as Degraded in about 21 s (median) and hard failures in about 85 s, a latency that is the deliberate cost of three-round hysteresis.

**Limitations.** The results come from one emulated cluster on a single host with small samples, and throughput is a synthetic estimate. The manager polls instead of watching, keeps hysteresis state in memory (lost on restart) and runs as a single replica. Degraded is not debounced, so short bursts of loss could make it flap. Probing is a full mesh, $O(N^2)$ per round. Only TCP is tested (no DNS, Service or application-level checks), and remediation is alert-only by design.

**Future work.** (1) Measure the interval from the start of a round instead of its end, which we estimate would cut hard-fault MTTD from about 85 s to about 55 s, and tune the thresholds per environment. (2) Sample pairs in large clusters. (3) Add separate conditions (for example ProbeReady and Stale) and Kubernetes Events on transitions. (4) Add a NetworkIntent API that encodes which paths policy is meant to block, so intended denials are not alarms. (5) Use eBPF flow data to corroborate probe evidence. (6) Evaluate on larger, multi-host clusters.

## IX. Reproducibility

Everything needed to reproduce the results is in the repository: https://github.com/nilay-v3rma/CS331-T011-KubernetesSelfHealing. Prerequisites are Docker, KIND 0.20 or later, kubectl, Go 1.22 or later, jq and Python 3. The commands below rebuild the testbed, run the unit tests and the full evaluation, and regenerate Tables V and VI.

```
git clone https://github.com/nilay-v3rma/
CS331-T011-KubernetesSelfHealing.git
cd CS331-T011-KubernetesSelfHealing
make test              # go vet + unit tests
make cluster-up        # KIND + Calico
make images load deploy
make deploy-sample     # NetworkHealthCheck
kubectl get nhc -w     # wait for Healthy
RESET=1 TRIALS=10 ./hack/run-p5-evaluation.sh
make deploy-monitoring # optional dashboard
```

`run-p5-evaluation.sh` runs F1–F6 (10 trials each by default; `RESET=1` starts a fresh CSV file), appends one row per trial to `docs/results/mttd-results.csv` and writes `docs/results/p5-summary.md` with `hack/summarize-results.py`. A single scenario can be measured with `make measure-mttd SCENARIO=F5 TRIALS=3`, and faults can be applied or cleared by hand with `make inject-fault` or `make clear-fault SCENARIO=Fx`. Two problems can appear on long-lived KIND clusters: if pods on one node hang in ContainerCreating with a Calico "Unauthorized" error, delete that node's calico-node pod; if the nodes cannot pull images, pull them on the host and load them with `kind load docker-image`.

## References


[1] The Kubernetes Authors, "Configure liveness, readiness and startup probes," Kubernetes Documentation. Accessed: Nov. 5, 2026. [Online]. Available: https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

[2] C. Guo et al., "Pingmesh: A large-scale system for data center network latency measurement and analysis," in Proc. ACM SIGCOMM, 2015, pp. 139–152, doi: 10.1145/2829988.2787496.

[3] C. Tan et al., "NetBouncer: Active device and link failure localization in data center networks," in Proc. 16th USENIX Symp. Networked Systems Design and Implementation (NSDI), Boston, MA, USA, 2019.

[4] The etcd Authors, "System limits," etcd Documentation, v3.5. Accessed: Nov. 5, 2026. [Online]. Available: https://etcd.io/docs/v3.5/dev-guide/limit/

[5] Bloomberg, "Debugging tool for Kubernetes which tests and displays connectivity between nodes in the cluster," GitHub repository. Accessed: Nov. 5, 2026. [Online]. Available: https://github.com/bloomberg/goldpinger

[6] The Prometheus Authors, "blackbox_exporter," GitHub repository. Accessed: Nov. 5, 2026. [Online]. Available: https://github.com/prometheus/blackbox_exporter

[7] The Kubernetes Authors, "API conventions," Kubernetes Community repository. Accessed: Nov. 5, 2026. [Online]. Available: https://github.com/kubernetes/community/blob/master/contributors/devel/sig-architecture/api-conventions.md

[8] The Kubernetes Authors, "Extend the Kubernetes API with CustomResourceDefinitions," Kubernetes Documentation. Accessed: Nov. 5, 2026. [Online]. Available: https://kubernetes.io/docs/tasks/extend-kubernetes/custom-resources/custom-resource-definitions/

[9] Kubernetes SIGs, "kind," Accessed: Nov. 5, 2026. [Online]. Available: https://kind.sigs.k8s.io/

[10] Tigera, "About Calico | Calico Documentation," Accessed: Nov. 5, 2026. [Online]. Available: https://docs.tigera.io/calico/latest/about/

[11] "tc-netem(8) — Linux manual page," Linux manual page. Accessed: Nov. 5, 2026. [Online]. Available: https://man7.org/linux/man-pages/man8/tc-netem.8.html

## Appendix A: Team Work Distribution and Member Rating

### *A.1 Work Distribution*

| # | Member Name | Roll No. | Tasks Done | Rating (1–10, unique) |
|---|---|---|---|---|
| 1 | Siddhesh Umarjee | 23110347 | Built the KIND + Calico testbed and reproducible setup/reset scripts. Implemented portable node/CNI/policy fault injection for F1–F6, separated F2/F3/F4, and made F5 produce a real Degraded result. Built the MTTD harness and summary script, ran the 60-trial evaluation, generated the CSV/results, added CI checks, and documented testbed limitations. | 8 |
| 2 | Nilay Verma | 23110219 | Implemented bounded Kubernetes status: 500-entry cap, truncation/total counts, degraded-pair list, and condition handling. Reworked Prometheus metrics, fixed the classification metric semantics, aligned the Grafana dashboard and manager Service/ServiceMonitor, and audited the README/docs against the actual code. Added status and metric-truth tests. | 9.5 |
| 3 | Patel Ridham Vijaykumar | 23110238 | Integration lead: merged the workstreams, resolved cross-component issues, ran vet/tests/build/deploy, and prepared the final demo/release. Implemented the interval gate, TCP-only probeType, removed inert topology/sampleDegree, wired failure/success thresholds into hysteresis, replaced destructive remediation with AlertOnly, wired manager authentication, and fixed status serialization. | 10 |
| 4 | Anurag Singh | 24110061 | Fixed the classifier: empty matrix -> NoData, added Degraded at >=5% loss, and preserved hard-failure precedence. Implemented and validated per-pair hysteresis/stabilization, added fixtures and tests for NoData, 40% loss, and flapping, and verified the confidence and classification rules. | 8.5 |
| 5 | Eshaan Jaiswal | 24110116 | Implemented the bidirectional TCP probe with payload + 1-byte acknowledgement, including timeouts, so an accept-then-close listener no longer passes. Added bearer-token authentication, CIDR allowlisting, metadata/loopback rejection, HTTP server timeouts, manager token wiring, agent Secret/manifests, and 401/403 probe tests. | 9 |

## Appendix B: AI Usage Details

### *B.1 Prompts*

| # | AI Tool | Purpose |
|---|---|---|
| 1 | DeepSeek V4.1 Flash (Provided by OpenCode) | As a search tool to identify SOTA and current solutions to this problem |
| 2 | GLM 5.3-Flash Free (Provided by OpenCode) | For coding and understanding the Ridham's part |
| 3 | Claude opus 5.5 | For co-coding and debugging Nilay's and Anurag's part |
| 4 | Github Copilot | For debugging siddhesh's part |
| 5 | Claude opus 5.5 | For report polishing and adding visuals |
| 6 | Google Antigravity : Claude Sonnet 4.x  and Gemini 3.8 Flash | For coding and debugging Eshaan's part |

Relevant Prompt queries and response .md files are shared on the github <AI Prompts> folder.